Implementation and Ramifications of TAMER-OP: a Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

Erik Mendes Novak

2026-01-28

Contents

1 Introduction **1**

2 Background mathematical information **1**
2.1 Posets as categories; persistence modules as functors . . . 1
2.2 1-parameter persistence . . . 3
2.3 Multiparameter persistence and why it is hard . . . 4
2.4 Two complementary viewpoints . . . 5
2.5 Survey of partial solutions . . . 6
2.6 Brief software landscape . . . 7
2.7 Notation and conventions . . . 7

3 Tame modules **8**
3.1 Why "tame" is the right hypothesis for computation . . . 8
3.2 Encoding posets and encoding maps . . . 9
3.3 Definitions of tameness in this framework . . . 10
3.4 Why non-abelianity fears are usually irrelevant in computation . . . 10
3.5 Fringe presentations as a computable interface . . . 12
3.6 What tameness buys you: mathematical consequences . . . 14
3.7 What tameness buys you: algorithmic consequences . . . 14
3.8 Bridge to implementation . . . 15

4 Overview of TAMER-OP **17**
4.1 Design goals stated mathematically . . . 17
4.2 The end-to-end pipeline . . . 17
4.3 Core mathematical objects in the API . . . 17
4.4 Architectural layering . . . 17
4.5 Reproducibility and correctness . . . 17

5 The presentation formats **17**
5.1 Philosophy: multiple equivalent front-ends . . . 17
5.2 Finite fringe presentations . . . 17
5.3 $\mathbb{Z}^n$ presentations . . . 17
5.4 $\mathbb{R}^n$ and piecewise-linear presentations . . . 17
5.5 Conversions between formats . . . 17
5.6 Validation and other sanity checks . . . 17
5.7 Serialization and I/O . . . 17

6 Finite encoding constructors **17**
6.1 The encoding problem . . . 17
6.2 Region/signature idea . . . 17
6.3 Encoding algorithms by ambient domain . . . 17

6.4 Constructing the finite poset structure . . . 17
6.5 Point location and evaluation map . . . 17
6.6 Building a common encoding for multiple modules . . . 17
6.7 Complexity and failure modes . . . 17
6.8 Correctness and diagnostics . . . 17

7 Invariants **17**
7.1 Meta principle: invariants factor through the encoding . . . 17
7.2 Dimension-type invariants . . . 17
7.3 Rank-type invariants . . . 17
7.4 Mobius inversions and signed measures . . . 17
7.5 Slice-based invariants and distances . . . 17
7.6 Vectorizations and kernels . . . 17
7.7 Homological invariants derived from resolutions . . . 17
7.8 Geometry-aware invariants for when $Q = \mathbb{R}^n$ . . . 17
7.9 Stability and robustness discussion . . . 17
7.10 Practical API summary . . . 17

8 Homological Algebra and Derived Functors **17**
8.1 The Abelian Category of P-modules . . . 17
8.2 Projective and Injective Objects . . . 17
8.3 Resolutions . . . 17
8.4 Ext and Tor: definitions and computational model . . . 17
8.5 Multiplicative structures and higher structure . . . 17
8.6 Double complexes and spectral sequences . . . 17
8.7 The ergonomics of derived categories . . . 17
8.8 Relationship to multigraded commutative algebra . . . 17
8.9 What you use all this for in TDA . . . 17

9 Transport and comparison **17**
9.1 Change of posets as functorial transport . . . 17
9.2 Comparing modules on different encodings . . . 17
9.3 Comparison via invariants . . . 17
9.4 Interoperability with multipers and RIVET . . . 17
9.5 Case studies . . . 17

10 Statistics **17**
10.1 Data model for experiments . . . 17
10.2 Feature extraction recipes . . . 17
10.3 Basic inferential statistics . . . 17
10.4 Learning-oriented workflows . . . 17
10.5 Diagnostic plots . . . 17
10.6 Reproducibility and reporting . . . 17

11 Speed optimizations **17**
11.1 Where time is spent . . . 17
11.2 Tradeoffs in exactness versus speed . . . 17
11.3 Sparsity-aware algorithms . . . 17
11.4 Caching and memoization strategies . . . 17
11.5 Parallelism . . . 17
11.6 Geometry backend: controlling combinatorial explosion . . . 17

12 Benchmarks **17**
12.1 Philosophy and reproducibility . . . 17
12.2 Microbenchmarks (unit tests) . . . 17
12.3 End-to-end benchmarks . . . 17
12.4 Comparisons with multipers and RIVET . . . 17

12.5 Discussion of results 17

13 Conclusion **17**

1 Introduction

2 Background mathematical information

Persistent homology assigns to data a family of homology groups indexed by a (possibly multi-dimensional) parameter, together with linear maps relating the groups as the parameter increases. The resulting algebraic object is a *persistence module*. In one parameter, persistence modules admit a clean structure theory (barcodes) and robust metrics; in multiple parameters, the same classification program becomes substantially more subtle. This section records the minimal language needed to state the objects studied in this thesis and to motivate the computational strategy pursued by TAMER-OP.

Throughout, fix a coefficient field $\Bbbk$ unless explicitly stated otherwise. When we discuss implementation choices later, we will emphasize why exact arithmetic over $\mathbb{Q}$ is often preferred for certifying ranks and homological invariants (cf. §2.7).

2.1 Posets as categories; persistence modules as functors

Posets and thin categories

Definition 1 (Poset)**.** A *partially ordered set* (poset) is a set Q equipped with a binary relation $\leq$ that is reflexive ($q \leq q$), antisymmetric ($q \leq q'$ and $q' \leq q$ implies $q = q'$), and transitive ($q \leq q'$ and $q' \leq q''$ implies $q \leq q''$).

Definition 2 (Poset as a category)**.** Given a poset $(Q, \leq)$, define a (small) category, which we also denote by Q, whose objects are elements of Q, and such that for $q, q' \in Q$ there is a unique morphism $q \to q'$ if and only if $q \leq q'$, and no morphism otherwise. Composition is forced by transitivity, and identities are forced by reflexivity.

A category arising from a poset in this way is *thin*: between any two objects there is at most one morphism. This viewpoint is the standard bridge between order theory and functorial algebraic constructions.

Modules over a poset

Definition 3 (Poset module / persistence module)**.** Let $(Q, \leq)$ be a poset and let $\mathbf{Vect}_\Bbbk$ denote the category of $\Bbbk$-vector spaces and $\Bbbk$-linear maps. A *Q-module* (or *persistence module indexed by Q*) is a covariant functor

$$M \colon Q \longrightarrow \mathbf{Vect}_\Bbbk.$$

Concretely, this is the data of:

(i) a vector space $M(q)$ for each $q \in Q$, and

(ii) for each comparable pair $q \leq q'$, a linear *structure map*

$$M(q \leq q') \colon M(q) \to M(q'),$$

subject to the functoriality conditions

$$M(q \leq q) = \mathrm{id}_{M(q)} \qquad \text{and} \qquad M(q \leq q'') = M(q' \leq q'') \circ M(q \leq q') \text{ for } q \leq q' \leq q''.$$

The indexing poset Q will typically be $\mathbb{R}$ or $\mathbb{R}^n$ with the usual order(s), or $\mathbb{Z}^n$ with the product (coordinatewise) order. The degree q is sometimes called a *parameter value*, and $M(q)$ a *fiber* or *stalk*.

Definition 4 (Morphisms)**.** A morphism of Q-modules $f\colon M \to N$ is a natural transformation of functors. Equivalently, it is a family of linear maps $f_q\colon M(q) \to N(q)$ indexed by $q \in Q$ such that for every $q \le q'$ the following square commutes:

$$\begin{tikzcd} M(q) \arrow[r, "M(q\le q')"] \arrow[d, "f_q"'] & M(q') \arrow[d, "f_{q'}"] \\ N(q) \arrow[r, "N(q\le q')"] & N(q'). \end{tikzcd}$$

We write $\mathrm{Hom}_Q(M,N)$ for the $\Bbbk$-vector space of such natural transformations (when the fibers are finite-dimensional, $\mathrm{Hom}_Q(M,N)$ itself is often computable by linear algebra).

Functor categories are abelian

The category of $\Bbbk$-vector spaces is abelian, and functor categories into an abelian category remain abelian. In particular, Q-modules admit kernels, cokernels, images, and short exact sequences.

Proposition (Pointwise abelian structure)**.** *For any small category $\mathcal{C}$, the functor category $\mathbf{Vect}_\Bbbk^{\mathcal{C}} = \mathrm{Fun}(\mathcal{C}, \mathbf{Vect}_\Bbbk)$ is abelian. In particular, for $\mathcal{C} = Q$ a poset category, kernels and cokernels are computed* objectwise*: for a morphism $f\colon M \to N$ one has*

$$(\ker f)(q) = \ker(f_q), \qquad (\operatorname{coker} f)(q) = \operatorname{coker}(f_q),$$

and similarly for images and coimages, with the induced structure maps coming from functoriality.

Remark. Proposition 2.1 is conceptually important because it reduces many categorical constructions on persistence modules to familiar linear algebra at each parameter value. Much of TAMER-OP is built around exploiting this pointwise nature after reducing an a priori infinite indexing poset to a finite one.

Upsets, downsets, and indicator modules

A recurrent theme (central to Miller's theory of tameness) is that many computationally tractable persistence modules can be assembled from very simple "indicator" modules associated to order ideals and filters.

Definition 5 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* (order filter) is a subset $U \subseteq Q$ such that if $q \in U$ and $q \le q'$, then $q' \in U$.

(ii) A *downset* (order ideal) is a subset $D \subseteq Q$ such that if $q' \in D$ and $q \le q'$, then $q \in D$.

For $q \in Q$, the *principal upset* and *principal downset* are

$$\uparrow q := \{q' \in Q : q \le q'\}, \qquad \downarrow q := \{q' \in Q : q' \le q\}.$$

Definition 6 (Indicator modules)**.** Let $U \subseteq Q$ be an upset. The *upset indicator module* $\Bbbk[U]$ is the Q-module defined by

$$\Bbbk[U](q) := \begin{cases} \Bbbk & q \in U, \\ 0 & q \notin U, \end{cases}$$

and for $q \le q'$ the structure map $\Bbbk[U](q \le q')$ is the identity $\Bbbk \to \Bbbk$ when $q, q' \in U$ and is the unique zero map otherwise. Likewise, for a downset $D \subseteq Q$, define the *downset indicator module* $\Bbbk[D]$ by the same formula with D in place of U.

Remark. The closure property in Definition 25 is exactly what guarantees functoriality for the "identity within the region, zero outside" convention in Definition 26. For general subsets $S \subseteq Q$, the same prescription can fail to define a functor because composition along chains may force a map to be zero even when both endpoints lie in S.

Remark (Yoneda intuition). When Q is viewed as a category, the representable functor $\mathrm{Hom}_Q(q,-)$ takes values in sets; its $\Bbbk$-linearization yields precisely the principal upset indicator $\Bbbk[\uparrow q]$. Dually, the corepresentable $\mathrm{Hom}_Q(-,q)$ linearizes to $\Bbbk[\downarrow q]$. For finite Q, these principal indicator modules behave like indecomposable projectives and injectives in the functor category, and they serve as building blocks for resolutions (compare with [Mil20]).

2.2 1-parameter persistence

This subsection recalls the classical one-parameter story, both to establish notation and to emphasize what breaks in the multiparameter setting.

From filtrations to persistence modules

Definition 7 (Filtration)**.** Let Q be a poset. A *Q-filtration* of a topological space X is a functor

$$\mathcal{F}\colon Q \longrightarrow \mathbf{Top}$$

such that $\mathcal{F}(q)$ is a subspace of X for each q, and for $q \le q'$ the morphism $\mathcal{F}(q \le q')$ is the inclusion $\mathcal{F}(q) \hookrightarrow \mathcal{F}(q')$. Equivalently, $\mathcal{F}$ is an order-preserving assignment of subspaces.

Applying singular homology with coefficients in $\Bbbk$ yields a persistence module.

Definition 8 (Persistent homology module)**.** Given a Q-filtration $\mathcal{F}$ and an integer $i \ge 0$, the associated ith *persistent homology module* is the composite functor

$$H_i(\mathcal{F};\Bbbk)\colon Q \xrightarrow{\mathcal{F}} \mathbf{Top} \xrightarrow{H_i(-;\Bbbk)} \mathbf{Vect}_{\Bbbk}.$$

The case $Q = (\mathbb{R}, \le)$ (or $Q = \mathbb{Z}$) is the classical *one-parameter* setting. The filtration may arise, for example, from sublevel sets of a function $f\colon X \to \mathbb{R}$, where $\mathcal{F}(t) = f^{-1}((-\infty, t])$.

Interval modules and barcodes

The exceptional feature of one-parameter persistence is that (under mild finiteness hypotheses) modules admit a complete decomposition into interval summands.

Definition 9 (Interval module)**.** Let $I \subseteq \mathbb{R}$ be an interval (possibly infinite or half-open, depending on the chosen convention). The *interval module* $\Bbbk_I$ is the $\mathbb{R}$-module defined by

$$\Bbbk_I(t) = \begin{cases} \Bbbk & t \in I, \\ 0 & t \notin I, \end{cases}$$

with structure maps $\Bbbk_I(s \le t)$ equal to the identity on $\Bbbk$ when $s, t \in I$ and the zero map otherwise.

To state the classification theorem, one usually imposes a finiteness condition, such as *pointwise finite-dimensional* (p.f.d.), meaning $\dim_{\Bbbk} M(t) < \infty$ for all t, together with an appropriate tameness condition to rule out pathologies in the indexing by $\mathbb{R}$ (see [CB15, CDSGO16]).

Theorem 10 (Barcode decomposition, informal statement)**.** *Let M be a p.f.d. persistence module indexed by $\mathbb{R}$ satisfying a standard tameness condition (e.g. q-tameness). Then M decomposes as a direct sum of interval modules,*

$$M \cong \bigoplus_{\alpha \in A} \Bbbk_{I_\alpha},$$

and the multiset of intervals $\{I_\alpha\}$ is uniquely determined by M up to reordering. This multiset is the barcode *(or persistence diagram) of M.*

The earliest formulations of this result for discrete indexing and finitely generated modules go back to persistent homology in computational topology; see, for example, [ZC05, EH10]. For $\mathbb{R}$-indexed modules, a definitive representation-theoretic proof is given by Crawley-Boevey [CB15].

Stability and distances

A major reason barcodes are useful in applications is *stability*: small changes in input data yield small changes in the output barcode with respect to a natural metric. There are several equivalent (or closely related) metrics in one parameter, notably the bottleneck distance on persistence diagrams and the interleaving distance on persistence modules; see [CDSGO16, Les15] for systematic treatments.

For the purposes of this thesis, we will use one-parameter stability primarily as a benchmark: it is an example of a situation where both (i) a complete discrete invariant exists (Theorem 10) and (ii) that invariant interacts well with perturbations. Both features become substantially more complicated in multiple parameters.

2.3 Multiparameter persistence and why it is hard

Definition and basic examples

Definition 11 (Multiparameter persistence module)**.** Fix $n \geq 2$. A (real-parameter) *n-parameter persistence module* is a $\mathbb{R}^n$-module, i.e. a functor $M\colon (\mathbb{R}^n, \leq) \to \mathbf{Vect}_{\Bbbk}$, where the order is coordinatewise:

$$a = (a_1, \ldots, a_n) \leq b = (b_1, \ldots, b_n) \quad \iff \quad a_i \leq b_i \text{ for all } i.$$

Likewise, a *lattice* n-parameter persistence module is a $\mathbb{Z}^n$-module.

Such modules arise from *multifiltrations* $\mathcal{F}\colon \mathbb{R}^n \to \mathbf{Top}$, for instance from a function $f\colon X \to \mathbb{R}^n$ via $\mathcal{F}(a) = f^{-1}((-\infty, a_1] \times \cdots \times (-\infty, a_n])$, and then applying homology as in Definition 8.

Why barcodes do not generalize directly

When $n \geq 2$, the classification landscape changes qualitatively. There is no analog of Theorem 10 in which modules decompose uniquely into interval-like building blocks (rectangles, axis-aligned boxes, etc.). One reason is representation-theoretic: the category of multiparameter persistence modules (even with finite-dimensionality restrictions) contains families of indecomposables depending on continuous parameters and exhibits behavior described as "wild" in the sense of quiver representation theory [CZ09, Oud15].

At an informal level, "wild" means that the isomorphism classification problem is at least as complicated as classifying representations of arbitrary finite-dimensional algebras. Consequently, one does not expect a complete discrete invariant such as a barcode to exist in general.

Remark (What replaces barcodes?). In multiparameter persistence, one typically trades completeness for computability and stability: instead of a single complete invariant, one studies families of computable invariants (rank-based, homological, and/or slice-based) that are informative for specific tasks and data regimes. Much of the design philosophy of TAMER-OP is to build a flexible pipeline that supports such families in a uniform algebraic framework.

Invariants that still make sense

Even though complete classification fails, many useful invariants remain available.

Definition 12 (Rank invariant)**.** Let M be a Q-module. The *rank invariant* of M is the function

$$\rho_M\colon \{(q, q') \in Q \times Q : q \leq q'\} \longrightarrow \mathbb{Z}_{\geq 0}, \qquad \rho_M(q, q') := \operatorname{rank}\big(M(q \leq q')\big).$$

Definition 13 (Hilbert function / dimension function)**.** The *dimension function* (often called the *Hilbert function* in graded settings) of a Q-module M is

$$h_M\colon Q \to \mathbb{Z}_{\geq 0}, \qquad h_M(q) := \dim_{\Bbbk} M(q).$$

Remark. For $Q = \mathbb{N}^n$ and suitably finite modules, h_M is literally the Hilbert function of a multigraded module over a polynomial ring. For general posets, it is still a basic and often stable summary of size.

Another robust idea is to reduce to one parameter by restricting along monotone maps from a totally ordered set into Q.

Definition 14 (Restriction to a chain / slice)**.** Let C be a totally ordered set (often $C = \mathbb{R}$ or $\mathbb{Z}$) and let $\gamma\colon C \to Q$ be an order-preserving map. The *restriction* (or *slice*) of a Q-module M along γ is the C-module $\gamma^* M$ defined by composition:

$$\gamma^* M := M \circ \gamma\colon C \to \mathbf{Vect}_{\Bbbk}.$$

When C is totally ordered and γ is injective, the image of γ is a chain in Q.

The restrictions $\gamma^* M$ are one-parameter persistence modules, hence admit barcodes under the usual hypotheses. Slicing therefore packages multiparameter data into a family of one-parameter summaries; this idea is central in 2-parameter visualization tools such as RIVET [RIVET].

Finally, there are homological invariants arising from resolutions and derived functors (Ext, Tor) that encode multi-parameter interactions. These will be pursued systematically later in the thesis, but the guiding idea is already visible in Miller's program: introduce a finiteness condition strong enough to guarantee finite presentations and resolutions (and thus computable derived invariants), while weak enough to encompass common persistence modules arising from data [Mil20].

2.4 Two complementary viewpoints

Multiparameter persistence can be approached from at least two complementary perspectives. The computational architecture of TAMER-OP is explicitly designed to move between them.

Topological viewpoint: multifiltrations

At the source is topology: data are used to build a multifiltration $\mathcal{F}\colon \mathbb{R}^n \to \mathbf{Top}$ (or $\mathbb{Z}^n \to \mathbf{Top}$), and persistent homology applies a functor $H_i(-;\Bbbk)$ to obtain

$$M_i := H_i(\mathcal{F};\Bbbk)\colon \mathbb{R}^n \to \mathbf{Vect}_{\Bbbk}.$$

The topological viewpoint is advantageous for:

- motivating what invariants should mean geometrically (birth, death, and interactions across parameters);
- importing stability statements from the theory of interleavings and perturbation bounds; and
- making contact with geometric constructions of filtrations (sublevel sets, distance functions, Vietoris–Rips and Čech complexes, etc.; see [EH10]).

Algebraic viewpoint: multigraded commutative algebra

When the indexing poset is discrete and additive, persistence modules are closely related to multigraded modules over polynomial rings.

Proposition (Equivalence with multigraded modules (discrete setting))**.** *Let $n \geq 1$ and consider the poset $(\mathbb{N}^n, \leq)$ with coordinatewise order. Then the category of $\mathbb{N}^n$-modules $M\colon \mathbb{N}^n \to \mathbf{Vect}_{\Bbbk}$ is equivalent to the category of $\mathbb{N}^n$-graded modules over the polynomial ring $\Bbbk[x_1, \ldots, x_n]$ with degree-preserving homomorphisms.*

Remark (Sketch of correspondence). Given a persistence module M, form the graded $\Bbbk$-vector space $\bigoplus_{a \in \mathbb{N}^n} M(a)$ and let x_i act by the structure maps $M(a \leq a + e_i)\colon M(a) \to M(a + e_i)$. Functoriality ensures that the x_i commute. Conversely, a graded $\Bbbk[x_1, \ldots, x_n]$-module determines the structure maps by multiplication with monomials. See [CZ09, Oud15] for detailed treatments in the persistence context.

This algebraic viewpoint brings powerful tools—resolutions, Betti numbers, Hilbert series—but also reveals why classification becomes hard when $n \geq 2$: $\Bbbk[x_1, \ldots, x_n]$ is no longer a PID, and multigraded modules can have intricate syzygies and moduli.

Finite posets and incidence algebras

TAMER-OP often works with finite posets (encoding posets) obtained by compressing a large or continuous parameter space. For a finite poset P, P-modules can be viewed as finite-dimensional representations

of the incidence algebra of P (or of a quiver with commutativity relations determined by P). This provides both:

- a representation-theoretic interpretation of projectives/injectives and resolutions (cf. Remark 2.1), and
- an algorithmic opportunity: for finite P, most constructions reduce to finite linear algebra, and many "global" constraints can be enforced by working only along cover relations (Hasse edges).

2.5 Survey of partial solutions

Because multiparameter persistence resists complete classification, a large body of work focuses on *partial* invariants and practical pipelines. The following four themes recur throughout the literature and motivate the design choices of this thesis.

(i) Slicing and fibered barcodes

The restriction construction in Definition 14 reduces a multiparameter module to a one-parameter module, hence to a barcode. Varying the slice (for instance, varying a line of positive slope in $\mathbb{R}^2$) yields a family of barcodes that can be used for visualization or for defining distances by integrating (or taking suprema of) one-parameter distances. In the 2-parameter case, the *fibered barcode* viewpoint underlies the RIVET framework [RIVET].

From a computational standpoint, slicing has two key advantages:

(i) it leverages mature one-parameter algorithms, and

(ii) it provides stable summaries for applications even when the full module is too large to manipulate directly.

Its main limitation is incompleteness: different modules can have identical slices.

(ii) Rank-based invariants and Möbius inversion

The rank invariant (Definition 12) is one of the most basic multiparameter invariants. In one parameter, the rank function determines the barcode, and the barcode can be recovered from ranks by an inclusion–exclusion (Möbius inversion) formula. In higher parameters, an analogous inversion yields *signed* measures on axis-aligned rectangles (sometimes called *signed barcodes* or *rectangle measures*). These objects can be used as:

- sparse, computable summaries of the rank invariant on a chosen grid, and
- feature representations for statistics/learning pipelines (e.g. kernels on signed measures).

From the algebraic side, such Möbius inversions are naturally phrased on products of chains, and they fit well with finite encodings (cf. (iv) below).

(iii) Commutative algebra invariants via resolutions

Under the equivalence in Proposition 2.4, a $\mathbb{N}^n$-module becomes a multigraded $\Bbbk[x_1, \ldots, x_n]$-module, and one can study:

- minimal free resolutions and multigraded Betti numbers,
- Tor/Ext groups and their algebra structures,
- associated primes and socles, and
- Hilbert series and local cohomology.

These invariants detect multi-parameter interactions that rank functions and slices can miss. However, they require a finiteness condition strong enough to ensure that resolutions are finite in a computational sense. For real-parameter modules, the usual noetherian hypotheses fail, motivating Miller's theory of *tameness* as a replacement [Mil20].

(iv) Geometric/combinatorial encoding strategies

A guiding principle in this thesis is that many modules of interest are *piecewise constant* over the parameter space in a controlled way. An *encoding* compresses the indexing poset Q (often $\mathbb{R}^n$ or $\mathbb{Z}^n$) to a finite poset P so that the module factors through P and all subsequent computations may be carried out on the finite object.

In Miller's framework, tameness can be characterized in several equivalent ways, including (among others) the existence of a finite encoding and the existence of certain finite "fringe" presentations by upset and downset indicator modules [Mil20]. The computational moral is that once such a finite encoding is constructed, one can safely deploy the abelian and homological algebra of finite poset modules (kernels, images, Ext/Tor, resolutions) using finite linear algebra.

2.6 Brief software landscape

This thesis is not primarily a software survey, but it is useful to clarify how different toolchains emphasize different parts of the multiparameter landscape. We will return to more detailed comparisons in later sections, after the TAMER-OP data model and pipeline are established.

- **RIVET.** In the 2-parameter case, RIVET is centered on the fibered barcode: it supports interactive exploration of how one-parameter barcodes vary over a family of lines, and it provides infrastructure for rank-based visualization and related distances [RIVET].
- **multipers.** The multipers ecosystem (as commonly used in applied TDA) targets scalable computation of multiparameter summaries and feature maps (often slice-based, rank-based, or image/landscape-type constructions). Its emphasis is typically on practical pipelines rather than complete algebraic structure. (A precise feature-by-feature comparison depends on the chosen backends and will be addressed later.)
- **TAMER-OP.** TAMER-OP is designed around Miller's tame-module viewpoint: build or ingest a presentation over a large indexing poset, compress it via a finite encoding poset, and then compute a broad family of invariants (rank-based, homological/derived, geometric, and statistical) by systematically exploiting abelian and homological algebra over the finite encoding.

Remark (Setting expectations for comparisons). Comparisons are most meaningful when the underlying mathematical object is held fixed. For example, comparing fibered barcodes is natural when both toolchains work with 2-parameter modules and slice-based summaries. Comparing resolution-based invariants is meaningful when both toolchains expose (or can emulate) multigraded homological algebra. TAMER-OP's main distinguishing feature is that it treats *finite encodings* as first-class objects, so that many invariants can be computed uniformly once the encoding is in place.

2.7 Notation and conventions

Coefficients and exactness

Unless stated otherwise, modules take values in $\mathbf{Vect}_{\Bbbk}$ for a fixed field $\Bbbk$. In implementations, there is a persistent tradeoff between speed and certifiability:

- Over floating-point fields (e.g. $\mathbb{R}$), numerical linear algebra is fast but can introduce rank errors near degeneracies.
- Over exact fields (notably $\mathbb{Q}$), ranks, kernels, and Smith/row-reduction type computations are certifiable but may be more expensive.

TAMER-OP is designed so that exact computations over $\mathbb{Q}$ can be used when needed, especially for invariants derived from ranks and homological algebra (kernels, cokernels, Ext/Tor dimensions).

Orders on $\mathbb{Z}^n$ and $\mathbb{R}^n$

We use the coordinatewise order on $\mathbb{Z}^n$ and $\mathbb{R}^n$:

$$a \leq b \Leftrightarrow a_i \leq b_i \text{ for all } i.$$

When writing $a < b$, we mean $a \leq b$ and $a \neq b$. The standard basis vectors are $e_1, \ldots, e_n$.

For a finite poset P, we generally denote its order by $\leq$ as well; the intended meaning should be clear from context. When P is used as an encoding poset for an ambient parameter space, elements of P are sometimes called *regions*.

Birth/death terminology and (co)presentations

When working with indicator modules (Definition 26) and their linear combinations, we follow the "birth/death" intuition from persistence:

- Upsets model *birth* regions (where generators become present and persist upward in the order).
- Downsets model *death* constraints (where relations become enforced downward in the order).

A *presentation* will typically mean a cokernel of a map between free objects built from upset indicators, while a *copresentation* dually involves downset indicators. In Miller's terminology, *fringe presentations* mix these two directions and are tailored to tame modules over general posets [Mil20].

3 Tame modules

Multiparameter persistence is often presented as a story about functoriality: a multifiltration of spaces (or chain complexes) gives rise to a functor from a parameter poset into vector spaces, and this functor is the algebraic object one studies. The difficulty is that, once the parameter poset has dimension ≥ 2, there is no reasonable discrete classification of all such functors, even under strong finiteness conditions such as pointwise finite-dimensionality; in representation-theoretic language, the classification problem becomes "wild" in general [CZ09]. For computational purposes one therefore needs a *hypothesis of controlled complexity*—a notion of finiteness that is strong enough to guarantee that modules admit finite descriptions and are stable under the constructions one cares about, but flexible enough to encompass the examples arising in applications.

In this thesis we adopt Ezra Miller's framework of *tame modules over posets* [Mil20]. The guiding idea is that the module should have only finitely many distinct "regions of algebraic behavior" in parameter space, so that it can be represented by *finite data* after passing through a suitable *encoding*. A primary goal of TAMER-OP is to turn this philosophy into concrete data structures and algorithms; the present section sets up the mathematical backbone that makes such an implementation sensible.

3.1 Why "tame" is the right hypothesis for computation

The classical 1-parameter theory enjoys a powerful structure theorem: under mild hypotheses, persistence modules decompose as direct sums of interval modules, and the resulting multiset of intervals is encoded by a barcode or persistence diagram. This viewpoint underlies much of applied persistent homology, as well as practical algorithms for computing invariants from filtered data [ZC05]. One rigorous formulation of this classification is due to Crawley–Boevey: pointwise finite-dimensional persistence modules over a totally ordered index set admit interval decompositions under appropriate finiteness hypotheses [CB15].

The multiparameter setting behaves fundamentally differently. Even for $\mathbb{Z}^n$-indexed modules with $n \geq 2$, indecomposables occur in families and the category does not admit a discrete complete invariant analogous to a barcode [CZ09]. From a computational perspective, this means that one should not expect a *complete* classification of multiparameter persistence modules by a finite combinatorial object. Instead, one aims for a framework in which

(i) the objects of interest admit *finite representations*,

(ii) the relevant algebraic constructions (kernels/cokernels, derived functors, etc.) stay within the class, and

(iii) the invariants one computes are *functorial* and stable under natural operations on the input data.

Miller's notion of tameness is designed with exactly these desiderata in mind. Very roughly, a module is tame if it is constant on finitely many regions of parameter space and therefore can be expressed via a

finite "model poset" that records the comparability relations among these regions [Mil20]. The definition is arranged so that the category of tame modules (with an appropriate restriction on morphisms) is abelian, hence supports homological algebra; moreover, tameness admits several equivalent characterizations—combinatorial, topological, and homological—that are captured by the syzygy theorem (Theorem 31 below).

3.2 Encoding posets and encoding maps

The central finiteness notion used throughout this thesis is *finite poset encoding*. Intuitively, an encoding replaces a large (typically infinite) poset Q by a smaller poset P whose elements represent "regions" of the parameter space, such that the module on Q is obtained by pullback from a module on P.

Definition 15 (Poset encoding [Mil20, Def. 4.1])**.** Let Q be a poset, viewed as a category in the usual way. An *encoding* of a Q-module M by a poset P consists of

(i) an order-preserving map (equivalently, a functor) $\pi\colon Q \to P$, and

(ii) a P-module $H\colon P \to \mathbf{Vect}_{\Bbbk}$

together with an isomorphism of Q-modules

$$M \;\cong\; \pi^* H \;:=\; H \circ \pi.$$

The encoding is called *finite* if P has finitely many elements and each stalk $H(p)$ is a finite-dimensional $\Bbbk$-vector space.

Remark (Encodings as "finite models"). If $M \cong \pi^* H$, then M is constant on each fiber $\pi^{-1}(p)$: for any $q, q' \in Q$ with $\pi(q) = \pi(q') = p$, the only morphism $p \to p$ in the poset category is the identity, so the structure maps of M restrict to identities on the shared stalk $H(p)$. Thus a finite encoding exhibits M as having finitely many "constant regions" indexed by P. Conversely, in many geometric situations a partition of the parameter space into finitely many constant regions determines a finite encoding by taking P to be the poset of regions ordered by reachability (or by closure relations); see [Mil20] for the general equivalence among the various constructions.

The functorial viewpoint is especially important in what follows, so we record the basic formalism.

Definition 16 (Pullback along a poset morphism)**.** Let $\pi\colon Q \to P$ be a poset morphism. Precomposition defines a functor

$$\pi^*\colon \mathbf{Vect}_{\Bbbk}^P \longrightarrow \mathbf{Vect}_{\Bbbk}^Q, \qquad H \longmapsto H \circ \pi.$$

We refer to π^* as the *pullback* (or restriction) functor along π.

Lemma 17 (Exactness of pullback)**.** *For any poset morphism $\pi\colon Q \to P$, the pullback functor π^* is exact. In particular, it preserves finite limits and colimits (kernels, cokernels, and finite direct sums).*

Proof. The category $\mathbf{Vect}_{\Bbbk}^P$ of P-modules is a functor category with values in $\mathbf{Vect}_{\Bbbk}$, hence limits and colimits are computed pointwise. Given a short exact sequence $0 \to H' \to H \to H'' \to 0$ of P-modules, evaluating at any $p \in P$ yields a short exact sequence of vector spaces. Precomposing with π and then evaluating at $q \in Q$ simply evaluates the original sequence at $\pi(q)$, so exactness is preserved at every stalk, hence globally. □

Definition 15 formalizes the idea that the module "factors through" a finite poset. The following terminology, also standard in [Mil20], is useful when one wishes to emphasize the dependence on a fixed encoding map.

Definition 18 (Subordinate encoding [Mil20, Def. 4.7])**.** An encoding map $\pi\colon Q \to P$ is said to be *subordinate* to a Q-module M if there exists a P-module H such that $M \cong \pi^* H$.

Example 19 (Indicator modules admit trivial finite encodings [Mil20, Ex. 4.6])**.** Let $U \subseteq Q$ be an upset (upward-closed set). Then the indicator module $\Bbbk[U]$ (defined in Subsection 3.5) admits a finite encoding by the two-element chain $0 < 1$ via the map $\pi\colon Q \to \{0 < 1\}$ sending U to 1 and $Q \setminus U$ to 0; the corresponding module H has $H(1) = \Bbbk$ and $H(0) = 0$. Dually, any downset indicator module $\Bbbk[D]$ is encoded by the same chain with the roles of 0 and 1 reversed.

3.3 Definitions of tameness in this framework

The previous subsection defined what it means for a module to be encoded by a fixed poset P. Tameness is the assertion that such an encoding exists with P finite.

Definition 20 (P-tame and tame modules)**.** Fix a poset Q.

(i) Let P be a finite poset. A Q-module M is *P-tame* if there exists a poset morphism $\pi\colon Q \to P$ and a P-module H with finite-dimensional stalks such that $M \cong \pi^* H$.

(ii) A Q-module M is *tame* if it is P-tame for some finite poset P. Equivalently, M admits a finite poset encoding subordinate to it in the sense of Definition 15.

Remark (Comparison with other finiteness notions)*.* There are several finiteness conditions in the persistence literature, especially over $\mathbb{R}^n$ and $\mathbb{Z}^n$, including variants of "constructibility" and "q-tameness" based on finiteness of ranks of structure maps. Those conditions are well adapted to stability questions and to certain metric structures, but they do not by themselves supply a finite combinatorial model that supports the full suite of homological computations one might wish to perform. Finite encodings are a *stronger, more combinatorial* hypothesis: they guarantee that all information about M is carried by a finite poset P together with finite linear algebra on the stalks of a P-module. In particular, tameness is expressly designed to enable algorithmic computation in principle [Mil20].

A subtle point is that *not every* Q-module homomorphism between tame modules should be regarded as "computably tame". Allowing all homomorphisms can take one outside the tame world (for example, kernels of arbitrary maps need not be finitely encoded). The correct remedy is to restrict to morphisms compatible with a common finite description.

Definition 21 (Tame morphisms and the tame category [Mil20, Def. 4.27, Def. 4.29])**.** Let M, N be tame Q-modules. A homomorphism $\varphi\colon M \to N$ of Q-modules is called *tame* if there exists a finite "constant subdivision" of Q that is subordinate to both M and N, such that on each constant region the induced linear map is independent of the chosen parameter value (see [Mil20, Def. 4.27] for the precise formulation). The *category of tame Q-modules* is the category whose objects are tame Q-modules and whose morphisms are tame homomorphisms.

Theorem 22 (Tame modules form an abelian category [Mil20, Prop. 4.31])**.** *For any poset Q, the category of tame Q-modules (with tame morphisms) is abelian. In particular, it has kernels and cokernels, finite products and coproducts, and every monomorphism (resp. epimorphism) is a kernel (resp. cokernel).*

3.4 Why non-abelianity fears are usually irrelevant in computation

The worry voiced by many users of multiparameter persistence is the following. The ambient category $\mathbf{Vect}_{\Bbbk}^{Q}$ of Q-modules is abelian, so it is natural to form kernels, cokernels, homology, and derived functors there. However, the full subcollection of *tame* objects inside $\mathbf{Vect}_{\Bbbk}^{Q}$ is not automatically stable under these operations if one allows *arbitrary* natural transformations between tame objects. In other words, tameness can fail to be preserved if one performs abelian constructions in $\mathbf{Vect}_{\Bbbk}^{Q}$ without restricting the class of morphisms.

This subsection unpacks the mathematical content behind that concern and explains why, from the standpoint of computation, the bad behavior is almost never encountered. The key point is that there are two distinct notions of "morphism between tame modules", and only one of them is compatible with finite data structures.

Homomorphisms of tame objects versus tame morphisms

Recall from Remark **??** that a finite encoding $\pi\colon Q \to P$ converts a tame Q-module into a P-module for a finite poset P. In that finite setting, morphisms are ordinary natural transformations in $\mathbf{Vect}_{\Bbbk}^{P}$, hence are controlled by finitely many linear maps.

Inside $\mathbf{Vect}_{\Bbbk}^{Q}$, however, a natural transformation $\varphi\colon M \to N$ between tame modules can vary on infinitely many degrees even when M and N are each constant on finitely many regions. Miller isolates the maps that are compatible with finite encodings by requiring *constancy of the transformation on constant regions*: a homomorphism φ is called *tame* if there exists a finite constant subdivision subordinate to

both M and N such that on each region I the composite map $M_I \to M_q \to N_q \to N_I$ is independent of $q \in I$ [Mil20, Def. 4.27]. The category of tame modules uses these tame homomorphisms as its morphisms [Mil20, Def. 4.29 and Rem. 4.30]. The resulting category is abelian by Theorem 22.

A concrete failure mode

The distinction between arbitrary homomorphisms and tame morphisms is not cosmetic. The following example demonstrates that kernels can leave the tame world if the morphism is allowed to vary infinitely within a constant region.

Example 23 (A kernel of a homomorphism of tame modules need not be tame [Mil20, Ex. 4.25])**.** Let $Q = \mathbb{R}^2$ with the coordinatewise partial order, and let $L = \{(x, y) \in \mathbb{R}^2 : x + y = 1\}$ be the antidiagonal line. Let $U \subseteq Q$ be the closed upset above L, and let U° be its interior. Consider the tame modules

$$M = \Bbbk[U] \oplus \Bbbk[U], \qquad N = \Bbbk[U]/\Bbbk[U^\circ],$$

where N is supported on L. Define a surjection $\varphi\colon M \twoheadrightarrow N$ by taking φ to be zero on U° and, for each point $(x, y) \in L$, sending the two standard basis vectors of $M_{(x,y)} = \Bbbk^2$ to the scalars y and $-x$ in $N_{(x,y)} = \Bbbk$.

Then $K = \ker \varphi$ is *not* tame. Indeed, for $(x, y) \in L$, the stalk $K_{(x,y)}$ is the one-dimensional subspace of $\Bbbk^2$ spanned by (x, y), and these lines are pairwise distinct as (x, y) varies along L. Consequently, any constant subdivision subordinate to K must separate distinct points of L into distinct constant regions, forcing infinitely many regions.

The phenomenon in Example 23 is that M and N are each constant on a finite subdivision of Q (three regions: below L, on L, and above L), but the map φ is *not* constant on the region L itself. Hence φ is not a tame homomorphism, so there is no reason for its kernel to remain tame.

The tame category is closed under the usual constructions

Once morphisms are required to be tame, the basic abelian operations are safe.

Lemma 24 (Kernels and cokernels of tame homomorphisms are tame [Mil20, Lem. 4.28])**.** *Let $\varphi\colon M \to N$ be a tame homomorphism of Q-modules. Then $\ker \varphi$ and $\operatorname{coker} \varphi$ are tame modules, and the canonical maps $\ker \varphi \to M$ and $N \to \operatorname{coker} \varphi$ are tame morphisms. The same statement holds in the semialgebraic, PL, subanalytic, or class X settings when the constant subdivision witnessing tameness is of the corresponding type.*

Proof. Choose a finite constant subdivision of Q that is subordinate to M, N, and φ (as required in the definition of a tame homomorphism). For each constant region I, fix identifications $M_q \cong M_I$ and $N_q \cong N_I$ for all $q \in I$ so that the structure maps of M and N agree with the transition maps among the regionwise vector spaces. By tameness of φ, the induced linear map $M_I \to N_I$ is independent of $q \in I$.

Define regionwise vector spaces

$$K_I := \ker(M_I \to N_I), \qquad C_I := \operatorname{coker}(M_I \to N_I).$$

Using the structure maps between regions, these assemble into Q-modules K and C. By construction, $K = \ker \varphi$ and $C = \operatorname{coker} \varphi$ in the abelian category $\mathbf{Vect}_{\Bbbk}^{Q}$, and the chosen constant subdivision is subordinate to both K and C. Hence K and C are tame. The auxiliary hypotheses (PL, semialgebraic, etc.) are preserved because the same subdivision witnesses tameness in those settings. □

Lemma 24 is the formal version of the slogan that "computations with tame modules stay tame" as long as the maps used in the computation come from finite data. Example 23 shows that if one insists on allowing *all* homomorphisms between tame objects, then kernels and cokernels can escape the tame world, but those escaping morphisms are precisely the ones that do not admit a finite encoding.

Why the pathological maps do not appear in algorithms

Any explicit computation in this thesis ultimately proceeds by passing to some finite encoding $\pi\colon Q \to P$ and working in the finite functor category $\mathbf{Vect}_{\Bbbk}^{P}$. In that regime, every morphism is represented by finitely many linear maps, and pulling back along π produces a tame homomorphism in the sense of Definition 21. Exactness of pullback (Lemma 17) then implies that kernels and cokernels computed at the finite level remain finite after pulling back.

More explicitly, if $M \cong \pi^* H$ and $N \cong \pi^* G$ for P-modules H and G, and if $\bar{\varphi}\colon H \to G$ is a morphism in $\mathbf{Vect}_{\Bbbk}^{P}$, then $\varphi := \pi^*\bar{\varphi}$ is a tame homomorphism of Q-modules, and

$$\ker(\varphi) \cong \pi^*(\ker \bar{\varphi}), \qquad \operatorname{coker}(\varphi) \cong \pi^*(\operatorname{coker} \bar{\varphi}),$$

because π^* is exact. Therefore the outputs of the standard constructions (kernel, cokernel, homology of a complex, and derived functors computed via finite resolutions) remain tame whenever the computation is expressed at the level of a finite encoding. In particular, the "risky" behavior in Example 23 cannot occur unless one is willing to specify a morphism that varies on infinitely many degrees inside a constant region, which is not compatible with the finite data structures used for encoding and computation.

Recent results that restore full abelianity under geometric hypotheses

The preceding discussion explains why Miller restricts morphisms in the tame category. There is also a complementary line of results showing that, for many geometric classes of finite encodings, the restriction is in practice unnecessary: under suitable hypotheses on encoding fibers, every homomorphism between encodable modules can be realized inside a finite encoding.

One precise formulation is due to Waas [Waa24], who introduces a connectivity condition on encoding fibers ensuring that any finite set of encodable modules admits a *common* encoding that captures *all* morphisms between them [Waa24, Prop. 3.5]. In particular, the resulting class of encodable persistence modules forms a full abelian subcategory of $\mathbf{Vect}_{\Bbbk}^{Q}$ [Waa24, Thm. 1.1]. For parameter posets $Q = \mathbb{R}^n$, his hypotheses are satisfied by natural families of "geometric" regions built from topologically closed PL, semialgebraic, or subanalytic upsets (and their complements and finite unions).

Conceptually, the point is that the failure in Example 23 relies on large disconnected families of degrees (here the continuum of points on L) along which a map can vary independently. Connectivity hypotheses on encoding fibers rule out that degree of freedom, forcing morphisms to be controlled by finite combinatorial data. Waas also notes that one can alternatively prove the corresponding subanalytic statement by translating the problem into constructible sheaf theory and invoking results of Miller and Kashiwara-Schapira [Waa24, Rem. 3.19], [Mil23, KS18].

These developments give a precise mathematical interpretation of the informal claim that "if the input is tame, then the output is almost always tame": the non-tame outputs arise from morphisms that do not admit finite encodings, and such morphisms are excluded either (i) by working in the tame category (tame morphisms by definition), or (ii) by restricting attention to the geometric subclasses of tame modules for which all relevant morphisms are automatically encodable.

3.5 Fringe presentations as a computable interface

Finite encodings are the most conceptually direct way to impose combinatorial finiteness, but they are not always the most convenient format for *input* or for certain computations. Miller's framework supplies a complementary algebraic interface: *fringe presentations*. These generalize the idea of presenting a module via generators and relations, where generators are organized as upsets (birth regions) and relations as downsets (death regions).

We begin by recalling the relevant combinatorics of subsets of a poset.

Definition 25 (Upsets and downsets)**.** Let Q be a poset.

(i) An *upset* (or upper set) in Q is a subset $U \subseteq Q$ such that $q \in U$ and $q \leq q'$ imply $q' \in U$.

(ii) A *downset* (or lower set) in Q is a subset $D \subseteq Q$ such that $q \in D$ and $q' \leq q$ imply $q' \in D$.

For $q \in Q$, the *principal upset* generated by q is $\uparrow q := \{q' \in Q : q \leq q'\}$, and the *principal downset* is $\downarrow q := \{q' \in Q : q' \leq q\}$.

Definition 26 (Indicator modules for upsets and downsets)**.** Let Q be a poset and let $S \subseteq Q$. The *indicator module* (or *indicator Q-module*) $\Bbbk[S]$ is the Q-module defined on objects by

$$\Bbbk[S](q) := \begin{cases} \Bbbk & \text{if } q \in S, \\ 0 & \text{if } q \notin S, \end{cases}$$

and on morphisms by the rule that for $q \leq q'$ the structure map $\Bbbk[S](q \leq q') \colon \Bbbk[S](q) \to \Bbbk[S](q')$ is the identity $\Bbbk \to \Bbbk$ when both $q, q' \in S$, and is 0 otherwise. When $S = U$ is an upset, $\Bbbk[U]$ is called an *upset module*; when $S = D$ is a downset, $\Bbbk[D]$ is called a *downset module*.

Remark (Projectives and injectives on a finite poset)*.* When P is a finite poset, the category $\mathbf{Vect}_{\Bbbk}^{P}$ can be identified with the category of finite-dimensional representations of the Hasse quiver of P with commutativity relations (equivalently, with modules over the incidence algebra of P). In this setting, principal upset modules $\Bbbk[\uparrow p]$ are the indecomposable projective objects and principal downset modules $\Bbbk[\downarrow p]$ are the indecomposable injective objects. This is one reason indicator modules are the correct building blocks for computable homological algebra; see [Mil20] for a detailed development in the general poset setting.

The next definition packages a tame module using finitely many upsets and downsets together with a matrix over $\Bbbk$. This is the abstraction of "birth" and "death" data familiar from barcodes, but adapted to general posets.

Definition 27 (Fringe presentation [Mil20, Def. 3.16])**.** Let Q be a poset and let M be a Q-module. A *fringe presentation* of M consists of

(i) a direct sum $F = \bigoplus_{i \in I} \Bbbk[U_i]$ of upset modules,

(ii) a direct sum $E = \bigoplus_{j \in J} \Bbbk[D_j]$ of downset modules, and

(iii) a Q-module homomorphism $\varphi \colon F \to E$

such that $\operatorname{im}(\varphi) \cong M$ and the component maps $\Bbbk[U_i] \to \Bbbk[D_j]$ satisfy a mild connectivity condition (see [Mil20, Def. 3.14]). The fringe presentation is *finite* if the index sets I and J are finite.

Remark (Relation to classical presentations)*.* If $Q = \mathbb{Z}^n$ with the usual partial order, then Q-modules can be interpreted as $\mathbb{Z}^n$-graded modules over the polynomial ring $\Bbbk[x_1, \ldots, x_n]$ (Section 2.4). In that language, upset modules behave like "free/flat" pieces and downset modules behave like "injective" pieces; the map φ organizes generators (birth upsets) and relations (death downsets) in a way that is robust under changes of geometry and does not depend on choosing a coordinate-aligned region decomposition.

Fringe presentations become especially concrete once one writes the map φ in coordinates.

Definition 28 (Monomial matrix [Mil20, Def. 3.17])**.** Let $\varphi \colon F \to E$ be a *finite* fringe presentation with $F = \bigoplus_{i=1}^{m} \Bbbk[U_i]$ and $E = \bigoplus_{j=1}^{r} \Bbbk[D_j]$. Choosing the natural bases on the 1-dimensional stalks of each indicator summand, the map φ can be encoded by an $r \times m$ matrix $\Phi = (\Phi_{ji})$ with entries in $\Bbbk$, whose columns are indexed by the birth upsets U_i and whose rows are indexed by the death downsets D_j. We refer to Φ as a *monomial matrix* for φ.

Lemma 29 (Support condition for monomial matrices [Mil20, Prop. 3.18])**.** *With notation as in Definition 28, one necessarily has $\Phi_{ji} = 0$ whenever $U_i \cap D_j = \varnothing$. Conversely, any matrix $\Phi \in \Bbbk^{r \times m}$ satisfying $\Phi_{ji} = 0$ when $U_i \cap D_j = \varnothing$ determines a fringe presentation map $\varphi \colon F \to E$.*

Example 30 (A single interval in one parameter)**.** Let $Q = \mathbb{R}$ with its total order and consider the interval module $\Bbbk[a, b)$. This module can be realized as the image of a map from the upset module $\Bbbk[a, \infty)$ to the downset module $\Bbbk(-\infty, b)$, and the corresponding monomial matrix is the 1×1 matrix $[1]$. For a direct sum of intervals (a barcode), the matching of births to deaths is reflected by a monomial matrix that is a permutation matrix after choosing orderings of births and deaths; compare [Mil20, Ex. 3.19] and the interval decomposition theorem [CB15].

3.6 What tameness buys you: mathematical consequences

The principal reason tameness is a useful organizing principle is that it is not a single isolated definition but rather part of a web of equivalent finiteness conditions, each suited to different tasks (geometric modeling, algebraic presentation, homological computation). The main structural result is Miller's syzygy theorem.

Theorem 31 (Syzygy theorem [Mil20, Thm. 6.12])**.** *Let Q be a poset and let M be a Q-module. Then M is tame if and only if it admits (and hence admits all of) the following finite descriptions:*

(i) a finite poset encoding subordinate to M;

(ii) a finite fringe presentation of M;

(iii) finite resolutions by upset modules and by downset modules.

Moreover, the constructions can be chosen compatibly with suitable auxiliary geometric structures (e.g. semialgebraic or piecewise-linear settings) when Q sits inside a partially ordered real vector space, and tame morphisms lift to morphisms of the corresponding finite presentations/resolutions.

Remark (Why equivalences matter for software)*.* Theorem 31 has two practical interpretations:

(i) It legitimizes switching among data representations depending on what is being computed. One might build a module from geometric data (suggesting a region-based encoding), transform it to a fringe presentation to expose "birth/death" structure, and then compute derived invariants via an indicator resolution.

(ii) It explains why it is sensible to base a computational pipeline on finite encodings. If a module is tame, then *some* finite encoding exists, and any other finite representation can (in principle) be converted to an encoding suitable for computation.

TAMER-OP is designed around these equivalences: the library implements multiple entry points (fringe-style and region-style) and reduces computations to finite poset models.

A more elementary but extremely important consequence is that, once an encoding is fixed, many constructions can be carried out on the finite poset P and then transported back to Q.

Lemma 32 (Reduction along an encoding)**.** *Suppose $M \cong \pi^* H$ where $\pi\colon Q \to P$ is a finite encoding and H is a P-module. Then every structure map of M is obtained by applying H to the image under π: for $q \le q'$ in Q,*

$$M(q \le q') \;=\; H\big(\pi(q) \le \pi(q')\big).$$

In particular, any invariant of M that is defined functorially from the diagram of vector spaces and structure maps (for example, ranks of structure maps, limits/colimits, or derived functors computed from resolutions) can be computed from H on the finite poset P.

Proof. The identity $M = H \circ \pi$ is the definition of pullback. The final statement follows because π^* is exact (Lemma 17) and because the data of M is completely determined by the data of H together with π. □

3.7 What tameness buys you: algorithmic consequences

The abstract consequences above translate directly into a computational strategy.

Finite linear algebra. If P is finite and H is a P-module with finite-dimensional stalks, then H is specified by:

(i) a finite list of vector spaces $H(p)$, and

(ii) linear maps $H(p \le p')$ subject only to functoriality constraints.

In a finite poset, functoriality can be enforced by recording maps only along cover relations (the Hasse diagram) and composing along paths. This is precisely the kind of finite linear-algebraic data structure on which one can implement kernels, images, cokernels, ranks, and derived constructions.

Homological algebra in a finite abelian category. The abelian category $\mathbf{Vect}_{\Bbbk}^{P}$ has enough projectives and injectives, and its basic building blocks are the principal upset and downset modules (Remark 3.5). Consequently, one can compute projective and injective resolutions by explicit finite algorithms, and then compute Tor and Ext groups by linear algebra on chain complexes. When a Q-module is tame, Theorem 31 guarantees that such finite resolutions exist *upstairs* in Q as well (in the appropriate tame category), but the key computational reduction is that it suffices to resolve the encoded object on a finite poset model.

3.8 Bridge to implementation

The remainder of this thesis is concerned with turning the preceding theory into a usable computational toolkit. The next sections explain:

(i) which concrete finite presentations and encodings TAMER-OP supports,

(ii) how finite encoding posets are constructed from geometric or algebraic input,

(iii) how invariants and derived functors are computed on finite poset models, and

(iv) how these computations feed into a statistics pipeline for data analysis.

In short: tameness provides the *mathematical contract* that makes the entire data-ingestion-to-statistics pipeline finite and computable.

4 Overview of TAMER-OP

4.1 Design goals stated mathematically

4.2 The end-to-end pipeline

4.3 Core mathematical objects in the API

4.4 Architectural layering

4.5 Reproducibility and correctness

5 The presentation formats

5.1 Philosophy: multiple equivalent front-ends

5.2 Finite fringe presentations

5.3 $\mathbb{Z}^n$ presentations

5.4 $\mathbb{R}^n$ and piecewise-linear presentations

5.5 Conversions between formats

5.6 Validation and other sanity checks

5.7 Serialization and I/O

6 Finite encoding constructors

6.1 The encoding problem

6.2 Region/signature idea

6.3 Encoding algorithms by ambient domain

6.4 Constructing the finite poset structure

6.5 Point location and evaluation map

6.6 Building a common encoding for multiple modules

6.7 Complexity and failure modes

6.8 Correctness and diagnostics

7 Invariants

7.1 Meta principle: invariants factor through the encoding

7.2 Dimension-type invariants

7.3 Rank-type invariants

7.4 Mobius inversions and signed measures

7.5 Slice-based invariants and distances

7.6 Vectorizations and kernels

7.7 Homological invariants derived from resolutions

7.8 Geometry-aware invariants for when $Q = \mathbb{R}^n$

7.9 Stability and robustness discussion

7.10 Practical API summary

8 Homological Algebra and Derived Functors

8.1 The Abelian Category of P-modules

References

[Mil20] E. Miller. *Homological algebra of modules over posets.* arXiv:2008.00063, 2020.

[Waa24] L. Waas. *Notes on abelianity of categories of finitely encodable persistence modules.* arXiv:2407.08666, 2024.

[Mil23] E. Miller. *Stratifications of real vector spaces from constructible sheaves with conical microsupport. Journal of Applied and Computational Topology*, 7(3):473–489, 2023. doi:10.1007/s41468-023-00112-1.

[KS18] M. Kashiwara and P. Schapira. *Persistent homology and microlocal sheaf theory. Journal of Applied and Computational Topology*, 2(1–2):83–113, 2018. arXiv:1705.00955.

[CB15] W. Crawley-Boevey. *Decomposition of pointwise finite-dimensional persistence modules. Journal of Algebra and Its Applications*, 14(5):1550066, 2015. doi:10.1142/S0219498815500668.

[ZC05] A. Zomorodian and G. Carlsson. *Computing persistent homology. Discrete & Computational Geometry*, 33:249–274, 2005. doi:10.1007/s00454-004-1146-y.

[CZ09] G. Carlsson and A. Zomorodian. *The theory of multidimensional persistence. Discrete & Computational Geometry*, 42:71–93, 2009. doi:10.1007/s00454-009-9176-0.

[CEH07] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. *Stability of persistence diagrams. Discrete & Computational Geometry*, 37(1):103–120, 2007. doi:10.1007/s00454-006-1276-5.

[ELZ02] H. Edelsbrunner, D. Letscher, and A. Zomorodian. *Topological persistence and simplification. Discrete & Computational Geometry*, 28:511–533, 2002. doi:10.1007/s00454-002-2885-2.

[Les15] M. Lesnick. *The theory of the interleaving distance on multidimensional persistence modules. Foundations of Computational Mathematics*, 15(3):613–650, 2015. doi:10.1007/s10208-015-9255-y.

[LW15] M. Lesnick and M. Wright. *Interactive visualization of 2-D persistence modules.* arXiv:1512.00180, 2015.

[RIVET] M. Lesnick and M. Wright. *RIVET: The rank invariant visualization and exploration tool* (software). `https://github.com/rivetTDA/rivet`.

[LS24] D. Loiseaux and H. Schreiber. *multipers: Multiparameter persistence for machine learning. Journal of Open Source Software*, 9(103):6773, 2024. doi:10.21105/joss.06773.

[CDSGO16] F. Chazal, V. de Silva, M. Glisse, and S. Y. Oudot. *The Structure and Stability of Persistence Modules.* SpringerBriefs in Mathematics. Springer, 2016. doi:10.1007/978-3-319-42545-0.

[Oud15] S. Y. Oudot. *Persistence Theory: From Quiver Representations to Data Analysis.* Mathematical Surveys and Monographs, Vol. 209. American Mathematical Society, 2015.

[EH10] H. Edelsbrunner and J. L. Harer. *Computational Topology: An Introduction.* American Mathematical Society, 2010.

[Mac98] S. Mac Lane. *Categories for the Working Mathematician.* Graduate Texts in Mathematics, Vol. 5. Springer, 2nd edition, 1998. doi:10.1007/978-1-4757-4721-8.

[Wei94] C. A. Weibel. *An Introduction to Homological Algebra.* Cambridge Studies in Advanced Mathematics, Vol. 38. Cambridge University Press, 1994.

[Eis95] D. Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry.* Graduate Texts in Mathematics, Vol. 150. Springer, 1995. doi:10.1007/978-1-4612-5350-1.

[MS05] E. Miller and B. Sturmfels. *Combinatorial Commutative Algebra.* Graduate Texts in Mathematics, Vol. 227. Springer, 2005. doi:10.1007/b138602.